

SMART WATER METER

방울이 프로젝트 최종 보고서

YOLOX-s 2단계 모델로 수도 계량기를 읽고
사용량·요금·환경 영향을 알려주는 Android 앱

팀 MAD · Make A Difference

김재헌	정승호	노승원	유수빈
PM			

물-에너지 융합 포럼 대국민 아이디어 공모전 제출 · 결과 대기 중

GitHub
github.com/jaeheonki/smart-water-meter-kr

Android appId
com.jjang.banguli

운영 API
AWS EC2 · Docker · FastAPI

01 BACKGROUND

프로젝트 배경과 목표

가정에서 쓰는 물은 요금으로만 인식되지만, 실제로는 취수·정수·가압·하수 처리 전 과정에 전력이 들고 온수를 데우는 데 가스와 전기가 추가로 든다. 물 1㎥를 아끼면 연동된 에너지와 탄소도 함께 줄어든다. 그러나 수도 검침은 월 1회 사후 통보 방식이라 사용자가 매일의 사용 습관을 인지할 방법이 사실상 없다.

우리가 정의한 문제

확인 주기가 길다
월 1회 고지서로만 사용량을 알 수 있어 일상의 행동과 연결되지 않는다.

환산이 보이지 않는다
물 절약이 에너지·탄소 절감으로 이어진다는 사실이 전달되지 않는다.

기록이 지속되지 않는다
단순 숫자 표시 앱은 금세 방치되어 행동 변화까지 가지 못한다.

프로젝트 목표

스마트폰으로 수도 계량기를 촬영하면 검침값을 자동으로 읽어 기록하고, 이전 검침과의 차이로 사용량을 계산한 뒤 요금과 에너지·탄소로 환산해 보여주는 앱을 만든다. 캐릭터 **방울이**의 상태 변화와 코인 보상으로 매일 기록하는 습관을 유도해 **측정에서 행동까지** 이어지도록 하는 것이 목표다.

사용자 흐름

촬영 계량기 사진 → **자동 인식** YOLOX 2단계 → **확인·수정** 사용자 확정 → **기록** 서버 저장 → **리포트** 요금·에너지·탄소 → **보상** 코인·꾸미기

주제를 바꾼 배경

처음에는 채용공고 분석 서비스를 검토했다. 그러나 공고 데이터의 수집·재배포 라이선스와 상업적 이용 조건을 확인하는 과정에서 팀이 감당하기 어려운 제약이 확인되어, 데이터를 직접 수집할 수 있고 공개 국가통계로 환산 근거를 세울 수 있는 수도 계량기 주제로 전환했다. 이 판단은 이후 모델 선정에서도 라이선스를 1순위 기준으로 삼는 계기가 되었다.

근거 에너지·탄소 환산 계수는 공개 국가통계를 사용한다. 상수도 0.23kWh/㎥와 하수 0.45kWh/㎥는 공공데이터포털 자료, 전력 배출계수 0.4173kg CO_2/kWh는 온실가스종합정보센터(GIR) 2023년 확정치다. 물 사용량은 계량기 실측값이지만 에너지와 탄소는 **추정치**이며, 절대값보다 추세 확인이 목적이다.

02 DATASET

데이터 수집

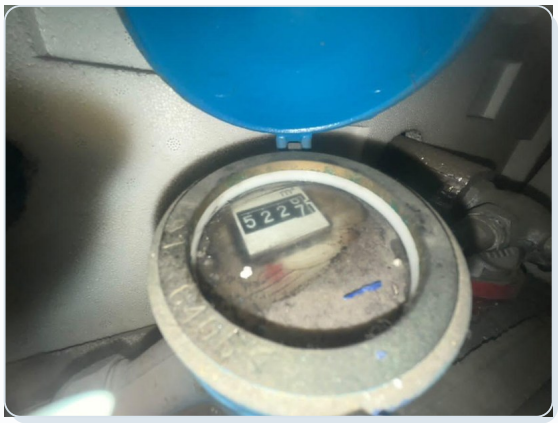
모델 학습에 쓸 수도 계량기 사진은 공개 데이터셋을 쓰지 않고 팀원 네 명이 직접 촬영했다. 2026년 5월 22일 아이디어 회의에서 1인당 50장 이상을 촬영해 데이터셋을 구성하기로 정했고, 길가와 아파트의 실제 계량기함을 열어 촬영했다. 그 결과 Model 1 학습용으로 약 200장을 확보했다.

촬영 인원 4명
각자 약 50장씩 분담해 한쪽 조건에 치우치지 않게 수집했다.

약 200장
Model 1 표시부 검출 학습에 사용한 계량기 원본 사진 규모다.

숫자 영역 라벨링
부가 표기와 단위는 제외하고 숫자 표시 영역만 라벨링했다.

직접 수집한 계량기 사진


노후 계량기 · 정면



계량기함 내부 · 그림자


근접 촬영 · 반사


각도 기울어짐

네 장의 사진만 봐도 촬영 각도, 조명, 유리면 반사, 먼지와 계량기 형태가 모두 다르다. 실제 사용 환경이 제각각이기 때문에 정면·균일 조명만 가정한 인식 방식으로는 대응하기 어렵다고 판단했고, 이후 파이프라인에 네 방향 회전 보정을 넣는 근거가 되었다.

한계 Model 2(개별 숫자 검출) 학습 데이터의 정확한 수량과 학습·검증 분할 기록은 저장소와 진행보고서에 남아 있지 않다. 따라서 이 보고서는 숫자 데이터 규모를 수치로 단정하지 않는다.

03 MODEL SELECTION

모델 실험과 선택

객체 탐지 모델 후보는 네 가지를 검토했다. 라이선스를 먼저 거르고, 학습이 성공한 모델만 성능을 비교했다. 공통 실험 조건은 COCO 포맷 데이터셋, 학습·검증 80:20 분할(seed 42), 50 epoch, 입력 640×640, 배치 8(T4 GPU)이다.

후보 모델과 진행 결과

모델	라이선스	진행 결과	판단
YOLOv8s	AGPL-3.0	학습 전 제외	배포 조건과 맞지 않아 학습 전에 제외
YOLOv9s	MIT	학습 시도 후 실패	학습 실패로 비교 후보에서 제외
YOLOX-Tiny	Apache 2.0	학습 시도 후 실패	성공한 가중치와 평가 결과 미확보
RF-DETR-nano	Apache 2.0	비교 평가 완료	검출 성능은 유사, 추론 시간 미측정
YOLOX-s	**Apache 2.0**	**최종 선정 · 현재 적용**	**추론 시간 실측, ONNX·FastAPI 연동 용이**

성능 비교

지표	YOLOX-s	RF-DETR-nano
mAP@0.50	1.000	1.000
mAP@0.50-95	0.729	0.743
추론 시간	**108.44 ms**	미측정

108.44ms의 정확한 의미

forward 96.44ms와 NMS 12.01ms의 합이다. **모델 선정 당시 표시부 검출 실험**에서 측정한 값으로, 현재 2단계 전체 파이프라인의 종단 간 속도가 아니다. 실행 장치와 평가 분할 정보가 남아 있지 않아 외부 벤치마크와 직접 비교할 수 없다.

주의 mAP@0.50이 두 모델 모두 1.000으로 나온 것은 모델이 완벽하다는 뜻이 아니다. 내부 검증 데이터의 규모가 작고 난이도가 낮았거나 학습·검증 분할이 유사했을 가능성을 먼저 의심해야 한다. 이 표는 **두 후보 중 무엇을 고를지 판단한 내부 실험 기록**으로만 사용한다.

YOLOX-s를 선택한 이유

라이선스

Apache 2.0으로 상업적 활용과 배포 조건이 자유롭다.

실측 속도 확보

유일하게 추론 시간을 측정해 배포 후 성능을 예측할 수 있었다.

ONNX 내보내기

PyTorch 없이 ONNX Runtime만으로 서버에서 실행할 수 있다.

FastAPI 연동

ONNX 세션을 재사용하는 형태로 API에 바로 붙일 수 있었다.

Exp 클래스 설정

입력 크기와 하이퍼파라미터가 한 파일에 모여 실험 반복이 쉬웠다.

top-1 검출 구조

이미지당 표시부 하나를 찾는 사용 사례에 그대로 맞았다.

04 PIPELINE

YOLOX-s 2단계 추론 파이프라인

단일 인식으로는 계량기 숫자를 안정적으로 읽지 못해 YOLOX-s 모델 두 개를 직렬로 연결했다. Model 1이 숫자 표시부의 위치를 찾고, Model 2가 잘라낸 표시부 안에서 숫자 0~9를 각각 검출한다. 촬영 방향을 사용자에게 강제하지 않기 위해 네 방향 회전을 모두 시도한 뒤 가장 좋은 방향을 고른다.



Model 1 · 어디를 읽을지

원본 사진 전체에서 숫자 표시부 하나만 찾는다. 배경과 계량기 몸체를 잘라내 다음 단계가 숫자에만 집중하게 만든다.

회전 보정 · 어느 방향인지

계량기함 안에서 촬영하면 사진이 돌아가는 경우가 잦다. 네 방향을 모두 시도하고 점수로 고르므로 사용자가 방향을 맞출 필요가 없다.

Model 2 · 무슨 숫자인지

잘라낸 표시부에서 숫자 0~9를 개별 검출하고, x좌표 순으로 이어 붙여 검침 숫자열을 만든다.

05 INFERENCE RESULT

실제 추론 결과

현재 저장소의 YOLOX 파이프라인을 CPU에서 실행해 직접 수집한 계량기 사진 네 장을 인식한 결과다. 네 장 모두 숫자열 반환에 성공했으며, 평균 신뢰도는 0.94에서 0.96 사이였다.



인식 숫자열	평균 신뢰도
5227	0.96

정면 · 노후 계량기



인식 숫자열	평균 신뢰도
0140	0.95

계량기함 내부 · 그림자



인식 숫자열	평균 신뢰도
0807	0.94

근접 촬영 · 유리면 반사



인식 숫자열	평균 신뢰도
0840	0.96

기울어진 각도

확인한 것

- 각도가 돌아간 사진에서도 네 방향 회전 보정으로 숫자열을 정상적으로 읽었다.
- 그림자와 유리면 반사가 있는 사진에서도 표시부 검출이 실패하지 않았다.
- 계량기 형태와 노후도가 서로 다른 네 대에서 모두 숫자열을 반환했다.

> **주의** 이 결과는 **제공된 검증 사진 4장 모두 숫자열 반환에 성공**했다는 사실까지만 말한다. 표본이 네 장이므로 일반적인 인식률이나 자릿수 정확도를 증명하지 않는다. 실제 인식률은 고정된 검증 세트를 만들어 자릿수 단위로 다시 측정해야 한다.

06 ARCHITECTURE

전체 시스템 구조

모델은 앱이 아니라 서버에서 실행한다. Android 앱은 사진을 찍어 HTTP로 보내고 결과를 받아 보여주는 외부 클라이언트이며, AWS EC2의 Docker 컨테이너 안에서 FastAPI와 YOLOX 두 모델, 계산 모듈이 동작한다.



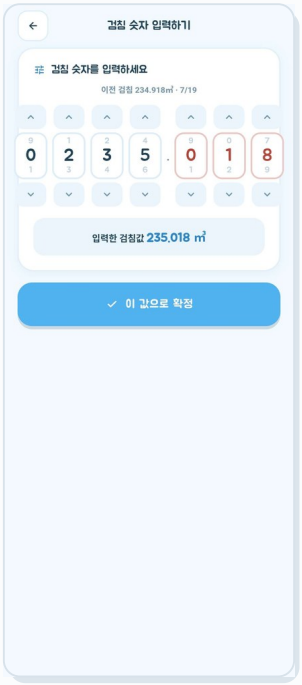
구성 요소	책임
Android 앱	촬영, 업로드, 인식 결과 확인·수정, 확정값과 리포트 표시
FastAPI	임시 파일 관리, 파이프라인 호출, 단위 변환, 기기별 검침 API 제공
YOLOX Model 1·2	숫자 표시부 검출과 표시부 내부 숫자 0~9 검출
계산 모듈	확정 검침값으로 사용량·요금·에너지·CO_2 계산
SQLite	기기별 검침 이력, 지역 설정과 계량기 교체 기준점 저장

Android 주요 기능 ① 홈과 검침 입력

홈 화면은 방울이의 상태와 연속 검침 기록을 보여주며, 검침은 하루 한 번을 기준으로 동작한다. 인식이 실패하거나 값이 틀렸을 때는 직접 입력으로 이어진다.



홈 화면



직접 입력 화면

실제 Galaxy 기기에서 촬영한 화면이며, 표시된 검침값과 사용량은 실제 가정의 소비량이 아니라 흐름 검증을 위해 입력한 테스트 데이터다.

홈 화면

- 방울이의 상태와 연속 검침 기록을 함께 보여줘 매일 여는 동기를 만든다.
- 검침을 이어가면 나무코인이 쌓이고, 코인으로 옷과 방 꾸미기 아이템을 적용한다.
- 오늘 기록이 있으면 신규 검침 버튼 대신 **오늘 기록 수정** 버튼이 나타난다.

검침 입력 규칙

상황	동작
오늘 첫 기록	POST /meter/manual로 새 검침 행을 저장한다.
당일 추가 입력	새 행을 만들지 않고 PUT /meter/today로 오늘 행을 수정한다.
직접 입력 화면	이전 검침값과 검침 날짜를 함께 안내해 잘못된 자릿수 입력을 줄인다.
현재값이 더 작음	422로 막고 값 확인과 계량기 교체 여부를 나눠 안내한다.
계량기 교체	이중 확인 후 새 기준값을 저장하고 기존 이력은 보존한다.

Android 주요 기능 ② 물 가계부와 에너지 리포트

확정된 검침값은 실제 기록 날짜를 기준으로 구간을 만들어 물 가계부에 쌓이고, 에너지 리포트에서 사용량·요금·에너지·탄소로 환산된다.



물 가계부



에너지 리포트

화면에 보이는 큰 수치는 실기기 흐름 검증을 위해 입력한 테스트 데이터이며, 실제 가정의 물 소비량이 아니다.

물 가계부

서버가 반환한 실제 기록 날짜 기준 구간을 최신순으로 보여준다. 구간별 사용량, 요금과 CO_2를 함께 표시하고, 기록이 없으면 검침을 유도하는 빈 상태를 보여준다.

에너지 리포트

주간 7일, 월간 30일, 연간 365일과 날짜 지정 기간을 지원한다. 선택한 기간과 바로 이전의 같은 길이 기간을 비교하고, 목표 사용량 대비 현황을 함께 보여준다.

물을 에너지와 탄소로 환산하는 방식

환산 경로	계수와 근거
온수 가열	온수량 × 비열 × 온도차 ÷ 보일러 효율 (가정용수 중 온수 약 45%, KISTI 실측)
상수도 공급	사용량 × 0.23 kWh/㎥ (한국수자원공사 정수 원단위)
하수 처리	사용량 × 0.45 kWh/㎥ (한국환경공단 하수 원단위)
탄소 환산	총 에너지 × 0.4173 kg CO_2/kWh (온실가스종합정보센터 2023 확정)
요금 계산	전국 17개 지역 요율 JSON과 누진 요금 공식 (조례·상수도사업본부 자료 기준)

추정 사용량은 계량기 실측값이지만 에너지와 탄소는 공개 원단위를 적용한 추정치다. 앱도 절대값을 단정하지 않고 기간 비교와 추세를 중심으로 표시한다.

API와 AWS 운영 구조

API는 계정 없이 기기 단위로 동작한다. 앱이 최초 실행 때 UUID를 만들어 로컬에 저장하고, 모든 요청에 **X-Device-ID** 헤더로 붙여 기기별 데이터를 분리한다.

주요 엔드포인트

메서드 · 경로	역할
POST /meter/scan	이미지를 YOLOX 파이프라인으로 분석해 검침값을 반환한다. DB에 저장하지 않는다.
POST /meter/manual	확정 검침값을 저장하고 사용량·요금·환경 영향을 계산한다. 하루 1건.
GET · PUT /meter/today	오늘 기록을 조회하고, 새 행 없이 같은 행을 수정한다.
GET /history · /history/period	구간별 이력과 지정 기간 이력을 반환한다.
GET /regions · /settings	지역 목록 조회와 기기별 지역 설정 저장·조회.
DELETE /device-data	현재 기기의 검침 이력과 설정만 삭제한다.
GET /health	컨테이너 상태 확인용. 내부와 Elastic IP 양쪽에서 점검한다.

서버 블로킹 문제와 해결

초기에는 async 엔드포인트 안에서 CPU 추론을 직접 호출했다. 추론이 수 초간 이벤트 루프를 점유해 그동안 다른 요청이 전부 지연됐다. 원인을 확인한 뒤 다음과 같이 정리했다.

추론을 스레드풀로 분리

run_in_threadpool로 파이프라인을 실행해 이벤트 루프가 막히지 않게 했다.

동시 추론 1건 제한

비차단 Lock으로 t2.micro의 CPU와 메모리를 넘어서는 동시 실행을 막았다.

중복 요청 즉시 응답

이미 추론 중이면 대기시키지 않고 503과 안내 문구를 바로 반환한다.

배포와 운영

항목	설정
실행 환경	EC2 Ubuntu 22.04, t2.micro 프리 티어, Docker 컨테이너 water-api
재시작 정책	unless-stopped. 인스턴스 재부팅 뒤 약 2분 안에 자동 복구를 실측했다.
볼륨	가중치와 SQLite를 호스트에 마운트해 컨테이너 교체와 무관하게 보존한다.
런타임 패키지	CPU 서버이므로 onnxruntime-gpu를 제외하고 onnxruntime을 설치한다.
헬스 체크	컨테이너 내부 localhost:8000/health와 외부 Elastic IP 양쪽을 확인한다.

운영 t2.micro는 프리 티어 기준 vCPU와 메모리가 빠듯해 이미지 빌드와 모델 실행 자원이 넉넉하지 않다. 동시 추론 제한으로 현재 검증 범위는 감당하지만, 사용자가 늘어나면 상위 인스턴스가 필요하다. 운영 API가 HTTP이므로 스토어 배포 전에는 HTTPS 전환도 필요하다.

10 VERIFICATION

테스트와 실기기 E2E 검증

자동화 테스트로 API 계약과 계산 로직, 프론트 어댑터를 검증하고, 마지막으로 실제 Galaxy 기기에서 앱 전체 흐름을 직접 확인했다.

56
Python pytest 통과
API 계약, 검침 규칙, 지역 요금과 환경 영향 계산

89
프론트 Node 테스트 통과
API 어댑터, 리포트 집계 함수, 화면 상태 전환

성공
Vite production build
Capacitor 동기화와 Android Debug APK 빌드까지 확인

실기기 Galaxy E2E 확인 항목

- [x] 앱을 실행하면 홈 화면으로 정상 진입한다.
- [x] 카메라 권한 요청이 표시되고 인앱 미리보기가 동작한다.
- [x] 실제 계량기 사진을 촬영해 서버로 전송한다.
- [x] YOLOX 인식 결과를 확인하고 값을 직접 수정할 수 있다.
- [x] 하루 한 번 신규 검침으로 저장된다.
- [x] 같은 날 다시 입력하면 새 행이 아니라 오늘 기록이 수정된다.
- [x] 저장한 검침이 물 가계부에 반영된다.
- [x] 저장한 검침이 에너지 리포트 집계에 반영된다.
- [x] 앱을 종료한 뒤 다시 실행해도 같은 기기 데이터로 홈 화면에 복귀한다.

범위 자동화 테스트는 실행 결과로 확인한 값이고, E2E 항목은 실제 Galaxy 기기에서 수행한 확인 결과다. 다만 이 검증은 앱 흐름이 끝까지 동작하는지를 본 것이며, 모델의 인식 정확도를 수치로 측정한 것은 아니다.

재현에 사용하는 검증 명령

실행 위치	명령	확인 대상
저장소 루트	uv run pytest	FastAPI 엔드포인트, 검침 규칙, 요금·환경 영향 계산
frontend	node --test	API 어댑터, 리포트 집계 함수, 화면 상태 전환
frontend	npm run build	Vite production 번들 생성
frontend	npx cap sync android	WebView 자산과 Camera·Notifications 플러그인 동기화
frontend/android	gradlew.bat assembleDebug	Android Debug APK 빌드

보고서에 적은 56건과 89건은 이 명령을 실행해 확인한 결과다. 모델 가중치가 없는 환경에서도 API 계약 테스트는 파이프라인 결과를 대체해 실행되므로, 저장소를 받은 팀원이 같은 명령으로 바로 재현할 수 있다.

11 CONCLUSION

결과, 한계와 향후 과제

공모전 제안서 단계의 아이디어를 실제로 동작하는 앱과 서버까지 구현했다. 제안서에서 계획했던 온디바이스 OCR 대신, 서버에서 YOLOX 두 모델을 실행하는 구조로 최종 구현했다.

완료한 결과

직접 수집한 데이터
팀원 4명이 촬영한 계량기 사진 약 200장으로 Model 1을 학습했다.

YOLOX-s 2단계 추론
표시부 검출과 숫자 검출을 직렬로 연결하고 네 방향 회전 보정을 넣었다.

ONNX Runtime 실행
PyTorch 없이 CPU 서버에서 추론 세션을 재사용한다.

FastAPI · EC2 Docker
기기별 검침 API를 EC2 컨테이너로 배포하고 자동 복구까지 확인했다.

React · Capacitor 앱
웹 UI를 그대로 Android 앱으로 빌드해 카메라와 알림을 연결했다.

실기기 E2E 성공
Galaxy 기기에서 촬영·인식·저장·리포트 전체 흐름을 확인했다.

한계와 향후 과제

구분	현재 한계	향후 과제
데이터	Model 2 학습 데이터의 수량과 분할 기록이 남아 있지 않다.	고정된 학습·검증 분할과 학습 설정을 다시 기록한다.
데이터	계량기 종류와 촬영 환경의 다양성이 제한적이다.	조명, 오염, 계량기 종류를 넓혀 추가 수집하고 재학습한다.
실험	모델별 학습 조건이 완전히 동일하지 않아 비교가 공정하지 않다.	동일 조건으로 후보 모델을 다시 학습해 비교한다.
평가	2단계 전체의 종단 간 인식률과 추론 시간을 측정하지 않았다.	자릿수 단위 정확도와 종단 간 지연 시간을 함께 측정한다.
운영	t2.micro는 빌드와 추론 자원이 빠듯하고 API가 HTTP다.	상위 인스턴스로 이전하고 도메인과 HTTPS를 적용한다.
플랫폼	iOS는 아직 구현하지 않았다.	Mac·Xcode 환경과 HTTPS API를 갖춘 뒤 iPhone 실기기로 검증한다.

참고 자료

저장소 github.com/jaeheonki/smart-water-meter-kr

Android appId com.jjang.banguli

저장소 문서 docs/yolox-model-pipeline.md · docs/api-guide.md · docs/deployment-guide.md · docs/frontend-integration-handoff.md

환산 계수 한국수자원공사 정수 원단위 · 한국환경공단 하수 원단위 · 온실가스종합정보센터 전력 배출계수(2023) · KISTI 가정용수 용도별 실측

팀 기록 아이디어 회의록(2026-05-22) · 정기 회의록(2026-06-12) · 방울이 제안서 · 프로젝트 진행 보고서